Application Demo

This notebook runs a demo of the context classification pipeline with a tuned YOLO model for Magic: The Gathering Arena.

Setup

```
import cv2
import tensorflow as tf
import numpy as np
from mss import mss
from ultralytics import YOLO
import time

context_classifier_model = "models/context_classifier.h5"
tuned_mtga_model = "models/yolo11s_tuned_52.pt"
```

Label Mapping

```
# Context classifier label mapping
label_mapping = {
    0: 'sol',
    1: 'mtga',
    2: 'mtgl',
    3: 'hs'
}

print("\nLabel mapping:")
for num, name in label_mapping.items():
    print(f"Class {num}: {name}")
```

```
 Label mapping:
 Class 0: sol
 Class 1: mtga
 Class 2: mtgl
 Class 3: hs
```

Demo

```
def process_frame(model, frame, context):
    img = cv2.cvtColor(np.array(frame), cv2.COLOR_BGRA2BGR)

    # Run inference
    if context == 1:
        results = model.predict(img, conf=0.5, verbose=False)

        # Draw bounding boxes
        for box in results[0].boxes:
            x_min, y_min, x_max, y_max = map(int, box.xyxy[0].tolist())
            class_id = int(box.cls[0])
            confidence = float(box.conf[0])

            # Draw rectangle
            cv2.rectangle(img, (x_min, y_min), (x_max, y_max), (0, 255, 0), 2)

            # Draw label
            label = f"{model.names[class_id]} {confidence:.2f}"
            cv2.putText(img, label, (x_min, y_min - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

    return img

def process_frame_context(model, screen):
    CONTEXT_IMG_SIZE = 128

    # Convert MSS screen to RGB for the model
    image = cv2.cvtColor(np.array(screen), cv2.COLOR_BGRA2RGB)

    image = tf.cast(image, tf.float32) / 255.0
```

```
    # Resize maintaining aspect ratio
    h, w = image.shape[0], image.shape[1]
    scale = CONTEXT_IMG_SIZE / max(h, w)
    new_h = int(h * scale)
    new_w = int(w * scale)
    image = tf.image.resize(image, [new_h, new_w])

    # Pad or crop to square 128x128
    image = tf.image.resize_with_crop_or_pad(image, CONTEXT_IMG_SIZE, CONTEXT_IMG_SIZE)

    # Convert back for preview
    preview = image.numpy() * 255.0
    preview = preview.astype('uint8')
    preview = cv2.cvtColor(preview, cv2.COLOR_RGB2BGR)

    cv2.namedWindow("Context Classifier Input", cv2.WINDOW_NORMAL)
    cv2.resizeWindow("Context Classifier Input", 640, 640)
    cv2.imshow("Context Classifier Input", preview)
    cv2.waitKey(1)

    # Add batch dimension for the model
    image_batch = tf.expand_dims(image, axis=0)

    # Get predictions
    start_time = time.time()
    predictions = model.predict(image_batch, verbose=0)
    inference_time = (time.time() - start_time) * 1000
    print(f"Context Inference Time: {inference_time:.2f} ms")
    predicted_class = np.argmax(predictions[0])

    return predicted_class


def live_labeling():
    mtga_model = YOLO(tuned_mtga_model)

    # Run context classifier inference
    context_model = tf.keras.models.load_model(context_classifier_model)

    # Screen capture configuration
    monitor = {"top": 0, "left": 0, "width": 1920, "height": 1080}
    sct = mss()

    cv2.namedWindow("Live Labels", cv2.WINDOW_NORMAL)
    cv2.resizeWindow("Live Labels", 960, 540)

    while True:
        # Capture screen
        screen = sct.grab(monitor)

        # Classify frame
        context = process_frame_context(context_model, screen)
        print("Context:", label_mapping[context])

        # Process frame
        labeled_frame = process_frame(mtga_model, screen, context)

        # Add context text to frame
        context_text = f"Context: {label_mapping[context]}"

        # Add black background rectangle
        text_size = cv2.getTextSize(context_text, cv2.FONT_HERSHEY_SIMPLEX, 1, 2)[0]
        cv2.rectangle(labeled_frame, (5, 5), (text_size[0] + 15, 40), (0, 0, 0), -1)
        cv2.putText(labeled_frame, context_text, (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

        # Display labeled frame
        cv2.imshow("Live Labels", labeled_frame)

        # Take screenshot
        if cv2.waitKey(1) & 0xFF == ord("s"):
            cv2.imwrite("images/screenshots/screenshot.png", labeled_frame)

        # Exit
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
```

```
    # Cleanup
    cv2.destroyAllWindows()

live_labeling()
```